PEMROGRAMAN BERORIENTASI OBJEK 2026

Exception Handling

Melaporkan kesalahan dengan jelas, menangkapnya di tempat yang tepat, dan tetap membereskan semuanya

PERTEMUAN

11

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Membedakan tiga jenis kesalahan program: sintaks/kompilasi, *runtime*, dan logika.
2. Membaca *traceback* (Python) dan *stack trace* (Java) untuk menemukan jenis, pesan, dan lokasi kesalahan.
3. Menangkap *exception* dengan `try`, beberapa penangkap dalam urutan yang benar, `else` (Python), dan `finally`.
4. Melempar *exception* untuk validasi, melempar ulang, dan menyimpan penyebab aslinya (*exception chaining*).
5. Membuat hierarki *exception* buatan sendiri yang membawa data tambahan.
6. Menjelaskan *checked* dan *unchecked exception* di Java beserta klausa `throws`, dan mengapa Python tidak memilikinya.

Mengingat Kembali



Dari pertemuan sebelumnya

- Pertemuan 04: *setter* dan konstruktor **menolak** data tidak valid dengan `raise ValueError(...)` / `throw new IllegalArgumentException(...)`.
- Pertemuan 05: `cari_menu()`/`cariMenu()` mengembalikan `None`/`null` jika kodenya tidak terdaftar.
- Pertemuan 07: pewarisan dan relasi *is-a*. Setiap jenis *exception* ternyata adalah kelas.
- Pertemuan 10: `proses()` dan `bayar()` mengembalikan benar/salah, jadi pemanggil harus rajin memeriksa sendiri.

Agenda Hari Ini

1. *Exception*: peluit wasit di dalam program
2. *Traceback* dan blok `try`
3. Banyak penangkap, melempar, dan *chaining*
4. *Checked* dan *unchecked* (khusus Java)
5. Studi kasus KedaiKita

BAGIAN 1

1

Exception: Peluit Wasit di dalam Program

Mengapa nilai khusus seperti `None`/`null` belum cukup untuk menandai kegagalan.

Kasir yang Bertemu Masalah

Analogi

- Dalam lima menit, kasir KedaiKita bertemu kode **M99** yang tidak ada, pesanan **50 porsi** padahal stok 10, dan **saldo QRIS** yang kurang.
- Sampai Pertemuan 10, kegagalan ditandai **nilai khusus** (`None`/`null`, `False`/`false`) yang mudah lupa dicek dan tidak menjelaskan alasan.
- Wasit sepak bola **meniup peluit**: permainan langsung berhenti, pelanggaran ditangani dulu, baru permainan berlanjut.

Sepak bola	Program
Wasit meniup peluit	*Exception* **dilempar**
Permainan langsung berhenti	Alur normal langsung berhenti
Pelanggaran ditangani dulu	*Exception* **ditangkap** dan ditangani
Catatan kecil di saku wasit	Nilai khusus seperti `None`/`null`

Tiga Jenis Kesalahan Program

Jenis	Kapan ketahuan	Contoh	Akibat
Sintaks/kompilasi	Sebelum program berjalan	lupa titik dua (Python), lupa `;` (Java)	Program sama sekali tidak bisa dijalankan
Saat berjalan (*runtime error*)	Saat program sedang berjalan	membagi dengan nol, mengubah teks `"dua"` menjadi angka	Program berhenti mendadak, **kecuali** ditangani
Logika	Tidak ketahuan dengan sendirinya	rumus diskon salah, `>` tertukar dengan `>=`	Program lancar, tetapi hasilnya salah

Exception handling mengurus jenis kedua. Kesalahan logika hanya bisa ditemukan lewat pengujian.

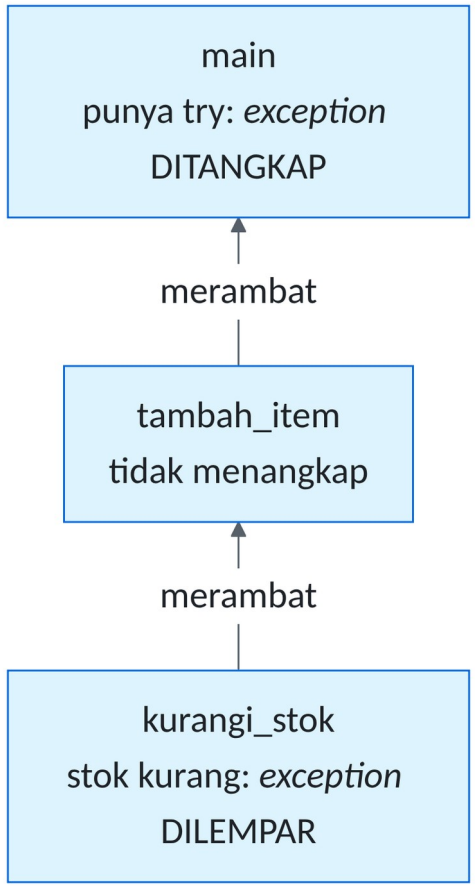
Exception: Sinyal yang Dilempar dan Ditangkap

Exception (pengecualian)

Objek yang mewakili kejadian tidak normal saat program berjalan. Saat masalah terjadi, *exception* **dilempar**; alur normal langsung berhenti, lalu *exception* mencari **penangkap** (*handler*) yang mau menanganinya.

- Melempar: `raise` (Python), `throw` (Java). Menangkap: `except` (Python), `catch` (Java).
- Karena objek, *exception* punya **kelas** (jenis kesalahan), **pesan**, dan boleh membawa **data tambahan**.

Exception Merambat ke Pemanggil



Baris yang tersisa di *method* yang dilewati tidak dijalankan. Jika `main` pun tidak menangkap, program berhenti.

BAGIAN 2

2

Traceback dan Blok `try`

Apa yang terjadi jika tidak ada yang menangkap, dan cara menangkap *exception*.

Membaca *Traceback* dan *Stack Trace*

Python: *traceback*, baca dari bawah ke atas

```
Traceback (most recent call last):
  File "patungan.py", line 11, in <module>
    main()
  File "patungan.py", line 7, in main
    hasil = hitung_patungan(120000, 0)
  File "patungan.py", line 2, in hitung_patungan
    return total // jumlah_orang
ZeroDivisionError: integer division or modulo by zero
```

3 titik awal program

2 tempat *exception* lahir

1 jenis dan pesan

Java: *stack trace*, baca dari atas ke bawah

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Patungan.hitungPatungan(Patungan.java:3)
    at Patungan.main(Patungan.java:8)
```

1 jenis dan pesan

2 tempat *exception* lahir

3 titik awal program

Python dibaca dari bawah ke atas, Java dari atas ke bawah. Cari baris pertama yang menyebut *file* milikmu.

Meminjam Kunci Laboratorium

Analogi

- Kamu meminjam **kunci laboratorium** di bagian tata usaha untuk praktikum.
- Praktikum bisa berjalan lancar, bisa juga terganggu, misalnya karena komputer mati.
- Apa pun yang terjadi, kunci **selalu** dikembalikan ke tata usaha.

Meminjam kunci lab	Blok di program
Coba jalankan praktikum	`try`
Jika komputer mati, pindah ke cadangan	`except` (Python) / `catch` (Java)
Jika lancar, kumpulkan laporan	`else` (khusus Python)
Apa pun yang terjadi, kunci dikembalikan	`finally`

Menangkap Pembagian dengan Nol

Baris yang gagal melempar *exception*, sisa blok `try` dilewati, lalu penangkap yang **jenisnya cocok** dijalankan.

Python python/contoh/contoh01_pembagian_nol.py

```python
def hitung_patungan(total: int, jumlah_orang: int) -> int:
    return total // jumlah_orang
# ... (dilewati)
try:
    print("Menghitung patungan untuk 0 orang...")
    hasil = hitung_patungan(120000, 0)
    print(f"Hasil: Rp{hasil} per orang")
except ZeroDivisionError:
    print("Gagal: jumlah orang tidak boleh nol")
```

Java java/contoh/Contoh01PembagianNol.java

```java
static int hitungPatungan(int total, int jumlahOrang) {
    return total / jumlahOrang;
}
// ... (dilewati)
try {
    System.out.println("Menghitung patungan untuk 0 orang...");
    int hasil = hitungPatungan(120000, 0);
    System.out.println("Hasil: Rp" + hasil + " per orang");
} catch (ArithmeticException e) {
    System.out.println("Gagal: jumlah orang tidak boleh nol");
}
```

`else` dan `finally`: Proyektor Selalu Dimatikan

Python python/contoh/contoh05_else_dan_finally.py

```python
def presentasi(proyektor: Proyektor, nama_file: str):
    proyektor.nyalakan()
    try:
        proyektor.tampilkan(nama_file)
    except ValueError as e:
        print(f"  Gagal presentasi: {e}")
    else:
        print("  Presentasi lancar, terima kasih!")
    finally:
        proyektor.matikan()
# ... (dilewati)
presentasi(proyektor, "laporan-akhir.docx")
```

Output

```
Proyektor R301 menyala
Gagal presentasi: file laporan-akhir.docx bukan PDF
Proyektor R301 dimatikan
```

Khusus Python

Blok `else` hanya ada di Python. Di Java, kodenya ditulis di akhir blok `try`. `finally` sama di kedua bahasa.

BAGIAN 3

3

Banyak Penangkap, Melempar, dan *Chaining*

Urutan penangkap, melempar *exception* sendiri, dan menyimpan penyebab aslinya.

Banyak Penangkap: Spesifik di Atas, Umum di Bawah

- Penangkap diperiksa **dari atas ke bawah**; hanya penangkap **pertama yang cocok** yang dijalankan.
- Cocok artinya jenisnya **sama** atau **turunan** jenis di penangkap (relasi *is-a*).
- Penangkap umum `Exception` ditulis **paling bawah** sebagai jaring pengaman terakhir.
- Beberapa jenis, satu penanganan: `except (ValueError, IndexError):` / `catch (A | B e)`.

Perhatikan

Jika urutannya terbalik, Python menerimanya tanpa protes (*bug* diam-diam), sedangkan Java **gagal dikompilasi**: `has already been caught`.

Melempar, Melempar Ulang, dan *Chaining*

Kebutuhan	Python	Java
Melempar *exception* baru	`raise ValueError("pesan")`	`throw new IllegalArgumentException("pesan");`
Melempar ulang yang sedang ditangani	`raise`	`throw e;`
Melempar baru + menyimpan penyebab	`raise TransferGagalError("pesan") from e`	`throw new TransferGagalException("pesan", e);`
Membaca penyebab	`e.__cause__`	`e.getCause()`

Validasi lebih awal (*fail fast*): tolak data yang salah secepat mungkin, sebelum objek berubah, agar data lama tetap utuh.

Exception Buatan Sendiri dan *Chaining*

`RekeningError` menjadi induk. Saldo kurang dibungkus menjadi transfer gagal, dan penyebab aslinya tetap tersimpan.

Python python/contoh/contoh07_exception_buatan_sendiri.py

```python
class RekeningError(Exception):
    # ... (dilewati)
class SaldoTidakCukupError(RekeningError):
    # ... (dilewati)
class TransferGagalError(RekeningError):
    # ... (dilewati)
def transfer(asal: Rekening, tujuan: Rekening, jumlah: int):
    try:
        asal.tarik(jumlah)
    except SaldoTidakCukupError as e:
        # ... (dilewati)
        raise TransferGagalError(pesan) from e
    tujuan.setor(jumlah)
```

Java java/contoh/Contoh07ExceptionBuatanSendiri.java

```java
class TransferGagalException extends RekeningException {
    // ... (dilewati)
}
// ... (dilewati)
static void transfer(Rekening asal, Rekening tujuan, int jumlah)
        throws TransferGagalException {
    try {
        asal.tarik(jumlah);
    } catch (SaldoTidakCukupException e) {
        // ... (dilewati)
        throw new TransferGagalException(pesan, e);
    }
    tujuan.setor(jumlah);
    // ... (dilewati)
}
```

BAGIAN 4

4

Checked dan *Unchecked* (Khusus Java)

Java membagi *exception* menjadi dua kelompok. Python tidak punya pembagian ini.

Checked dan *Unchecked Exception*

Aspek	*Checked*	*Unchecked*
Kelas	turunan `Exception`, **bukan** turunan `RuntimeException`	turunan `RuntimeException` (dan `Error`)
Contoh	`IOException`, `KedaiException` di KedaiKita	`ArithmeticException`, `NullPointerException`
Diperiksa *compiler*?	ya: "tangkap atau umumkan" dengan `throws`	tidak
Cocok untuk	situasi wajar yang harus diantisipasi: stok habis, *file* tidak ada	kesalahan pemrograman: indeks salah, variabel `null`

Python tidak punya pembagian ini; dokumentasikan *exception* yang bisa dilempar sebuah *method* di *docstring*-nya.

Wajib Ditangani: `KelasPenuhException`

Java java/contoh/Contoh08CheckedUnchecked.java

```java
class KelasPenuhException extends Exception {
    // ... (dilewati)
}
class KelasKuliah {
    // ... (dilewati)
    public void daftar(String nama) throws KelasPenuhException {
        if (peserta.size() >= kapasitas) {
            throw new KelasPenuhException(kode, kapasitas);
        }
        peserta.add(nama);
    }
}
// ... di dalam main
for (String nama : pendaftar) {
    try {
        pboA.daftar(nama);
    } catch (KelasPenuhException e) {
        System.out.println("Gagal: " + e.getMessage());
    }
}
```

Khusus Java

Method yang melempar *checked exception* wajib menulis `throws`; pemanggilnya wajib menangkap atau ikut mengumumkan.

STUDI KASUS



Studi Kasus KedaiKita

Semua tanda gagal lama diganti dengan *exception* khusus KedaiKita.

Apa yang Berubah di KedaiKita?

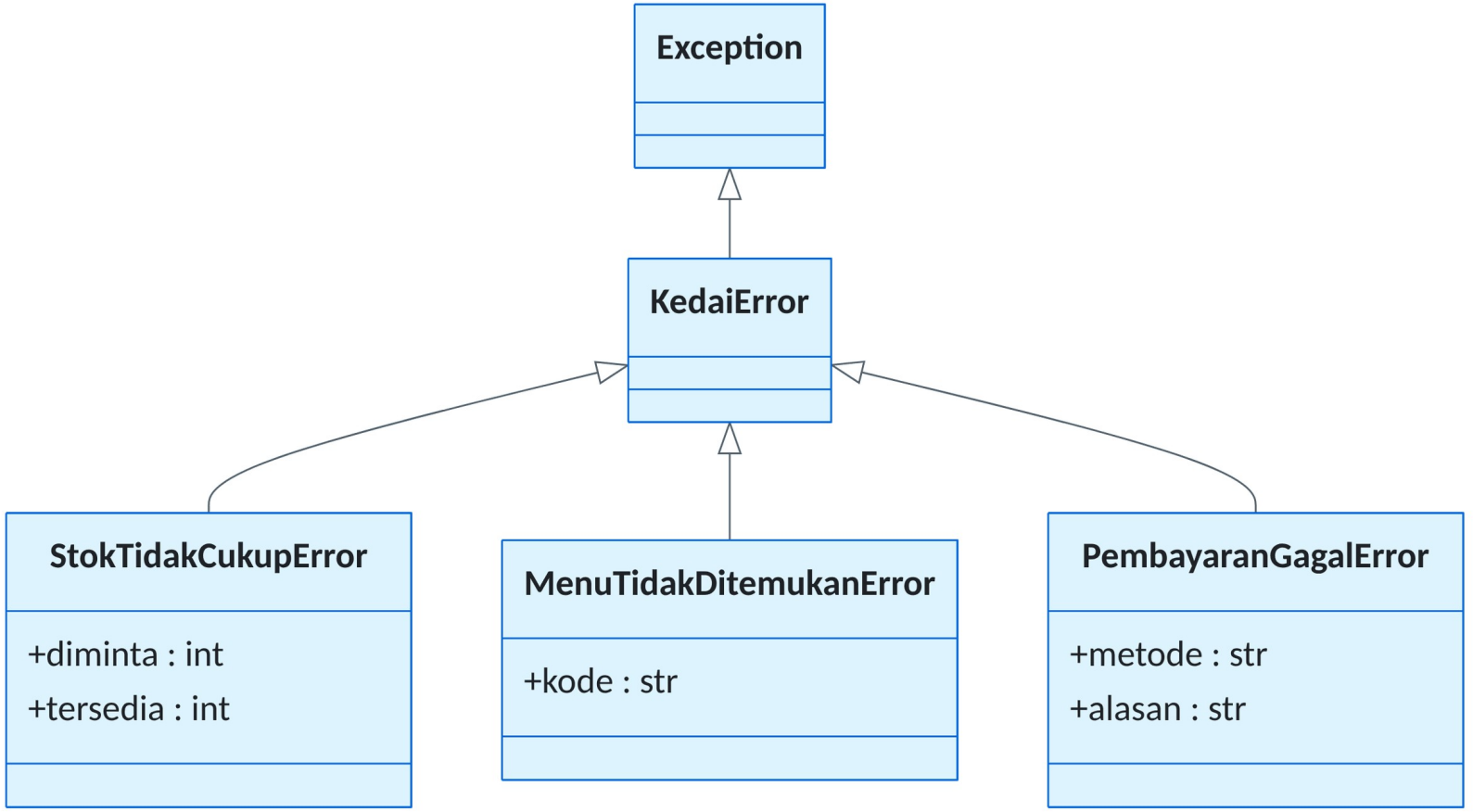
Studi kasus KedaiKita

Situasi	Pertemuan 10	Pertemuan 11
Kode menu tidak terdaftar	`cari_menu()`/`cariMenu()` mengembalikan `None`/`null`	melempar `MenuTidakDitemukanError`
Stok kurang	*exception* umum (`ValueError`/`IllegalArgumentException`)	melempar `StokTidakCukupError` + jumlah tersedia
Pembayaran gagal	`proses()` dan `bayar()` mengembalikan nilai salah	melempar `PembayaranGagalError` + metode dan alasan
Pesanan LUNAS diubah atau dibayar lagi	tidak dicegah	melempar induknya, `KedaiError`

Nama versi Python; versi Java berakhiran `Exception`, misalnya `KedaiException` dan `StokTidakCukupException`.

Hierarki *Exception* KedaiKita

Studi kasus KedaiKita



Semuanya turunan `KedaiError`, jadi satu penangkap induk bisa menangkap ketiganya sekaligus.

Penangkap Spesifik di Atas Induknya

Studi kasus KedaiKita

Lima permintaan Budi diproses satu per satu; setiap jenis kegagalan punya penangkapnya sendiri.

Python python/studi-kasus/main.py

```python
try:
    menu = kedai.cari_menu(kode)
    pesanan_budi.tambah_item(menu, jumlah)
except MenuTidakDitemukanError as e:
    print(f"  [GAGAL] {e}")
except StokTidakCukupError as e:
    print(f"  [GAGAL] {e}")
    # ... (dilewati)
except KedaiError as e:
    print(f"  [GAGAL] {e}")
except ValueError as e:
    # ... (dilewati)
else:
    print(f"  [OK] {jumlah} x {menu.nama}")
```

Java java/studi-kasus/Main.java

```java
try {
    Menu menu = kedai.cariMenu(kode);
    pesananBudi.tambahItem(menu, jumlah);
    // ... (dilewati)
} catch (MenuTidakDitemukanException e) {
    System.out.println("  [GAGAL] " + e.getMessage());
} catch (StokTidakCukupException e) {
    System.out.println("  [GAGAL] " + e.getMessage());
    // ... (dilewati)
} catch (KedaiException e) {
    System.out.println("  [GAGAL] " + e.getMessage());
} catch (IllegalArgumentException e) {
    // ... (dilewati)
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
"*Exception* itu sama dengan *bug*."	Tidak selalu. *Exception* bisa menandai situasi wajar seperti stok habis. Kesalahan logika malah sering tidak memunculkan *exception* sama sekali.
"Setelah *exception* ditangkap, program mengulang baris yang gagal."	Program melanjutkan ke baris setelah struktur `try`. Mencoba ulang harus ditulis sendiri, seperti pembayaran QRIS yang gagal lalu dicoba lagi dengan kartu kredit.
"`finally` hanya berjalan jika terjadi *error*."	`finally` **selalu** berjalan: saat berhasil, saat *exception* ditangkap, dan saat *exception* lanjut merambat ke pemanggil.

Kuis Cepat

Sebuah *exception* tidak tertangkap sehingga muncul *traceback* (Python) atau *stack trace* (Java). Di mana kamu mencari tempat *exception* itu **lahir**?

A Python: blok `File ...` paling atas; Java: baris `at ...` paling bawah

B Python: blok `File ...` paling bawah; Java: baris `at ...` paling atas

C Di kedua bahasa, selalu di baris pertama laporan

D Di kedua bahasa, selalu di baris terakhir laporan

Kuis Cepat: Jawaban

Sebuah *exception* tidak tertangkap sehingga muncul *traceback* (Python) atau *stack trace* (Java). Di mana kamu mencari tempat *exception* itu **lahir**?

A. Python: blok `File ...` paling atas; Java: baris `at ...` paling bawah

✓ **B.** Python: blok `File ...` paling bawah; Java: baris `at ...` paling atas

C. Di kedua bahasa, selalu di baris pertama laporan

D. Di kedua bahasa, selalu di baris terakhir laporan

Jawaban: B

Python menulis *most recent call last*, jadi tempat *exception* lahir ada di blok `File` paling bawah, tepat di atas baris `JenisError: pesan`. Java kebalikannya: baris `at` paling atas adalah tempat lahirnya, dan baris paling bawah adalah titik awal (`main`). Pilihan A tertukar; C dan D salah karena arah baca kedua bahasa berlawanan.

Kuis Cepat

Manakah kebiasaan yang **paling buruk** saat menangani *exception*?

A Menangkap *exception* di *method* yang tahu harus berbuat apa

B Penangkap kosong (`except Exception: pass` / `catch (Exception e) { }`) agar tidak terlihat *error*

C Memeriksa data sebelum mengubah atribut, lalu melempar *exception* jika datanya salah

D Mematikan atau menutup sumber daya di blok `finally`

Kuis Cepat: Jawaban

Manakah kebiasaan yang **paling buruk** saat menangani *exception*?

A. Menangkap *exception* di *method* yang tahu harus berbuat apa

✓ Penangkap kosong (`except Exception: pass` / `catch (Exception e) { }`) agar tidak terlihat *error*

C. Memeriksa data sebelum mengubah atribut, lalu melempar *exception* jika datanya salah

D. Mematikan atau menutup sumber daya di blok `finally`

Jawaban: B

Penangkap kosong "menelan" *exception*. Program terlihat lancar, tetapi hasilnya bisa salah tanpa jejak, seperti program yang mencetak "Suhu diatur ke 10 derajat" padahal perintahnya ditolak. Pilihan A, C, dan D justru praktik baik: menangkap di tempat yang tepat, validasi lebih awal, dan bersih-bersih di `finally`.

Ringkasan

- Tiga jenis kesalahan: sintaks/kompilasi, *runtime*, dan logika. *Exception handling* mengurus kesalahan *runtime*.
- ***Exception*** merambat ke pemanggil sampai ditangkap. Jika lolos, program berhenti dan mencetak laporan.
- *Traceback* Python dibaca dari bawah, sedangkan *stack trace* Java dibaca dari atas.
- `try` berisi kode berisiko, `except`/`catch` menanganinya, `finally` selalu berjalan, dan `else` khusus Python.
- Penangkap diperiksa dari atas ke bawah: yang spesifik di atas, yang umum di bawah.
- `raise`/`throw` melempar, `raise` atau `throw e;` melempar ulang, dan *chaining* menyimpan penyebab asli.
- Java membedakan *checked* (wajib ditangkap atau diumumkan dengan `throws`) dan *unchecked*; Python tidak.

Langkah Berikutnya

- Jalankan contoh di jalur pilihanmu (Python 01—07, Java 01—08) dan studi kasus KedaiKita.
- Kerjakan semua percobaan dan “Coba sendiri” di folder `PBO-latihan-saya/pertemuan-11`.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-11-Exception-Handling

Pindai untuk membuka folder pertemuan ini di GitHub